

Python + FastAPI: Roadmap

PHASE 1: Python Fundamentals + Dev Tooling (Weeks 1–4)

Week 1 — Syntax, Control Flow, Git & Terminal Basics

Concepts: Variables, data types, operators, if/elif/else, for/while loops, break/continue.

Tooling: terminal navigation, `git init/add/commit/push`, creating a GitHub repo, `.gitignore` basics **Test:**

- Write a script that prints all prime numbers between 1–100
- Commit it to a new GitHub repo with a proper commit message
- Q: What's the difference between `git add` and `git commit`?

Week 2 — Functions, Data Structures & Virtual Environments

Concepts: Functions, `*args/**kwargs`, scope, lists/tuples/dicts/sets, list comprehensions.

Tooling: `venv` (or `uv`), why you never install packages globally, `requirements.txt` **Test:**

- Build a contact book (dict-based) with add/search/delete, inside a proper venv with a `requirements.txt` generated via `pip freeze > requirements.txt` (exact version pins, not just `fastapi`)
- Q: Why does installing FastAPI globally cause problems later when you deploy?
- Q: Why pin exact dependency versions instead of using the latest? (A new major version dropping the day you redeploy can silently break your app)

Week 3 — File Handling & Error Handling

Concepts: Reading/writing files (txt, csv), try/except/finally, custom exceptions **Test:**

- Script that reads a CSV, filters rows, writes results to a new file, handles a missing-file error gracefully
- Q: What's wrong with a bare `except:` clause?

Week 4 — Object-Oriented Programming

Concepts: Classes, `__init__`, instance vs class attributes, inheritance, `__str__`/`__repr__`

Test:

- Build a `BankAccount` class with deposit/withdraw and a subclass `SavingsAccount` with interest
- Q: Difference between instance and class attributes?

Phase 1 Checkpoint: Command-line expense tracker (functions + class + file I/O + error handling), committed with a clean Git history showing incremental progress, not one giant commit.

PHASE 2: Web Basics + DSA Foundations (Weeks 5–6)

Week 5 — How the Web Works

Concepts: Client-server model, HTTP methods, status codes, JSON, REST principles, the `requests` library **Test:**

- Script that fetches data from a public API and prints a formatted summary
- Q: Difference between PUT and PATCH?

Week 6 — Big-O Basics & Intro DSA

Concepts: What Big-O means and why interviewers ask about it, time vs space complexity at a conceptual level (you don't need to derive proofs, just reason about it) **Test:**

- Solve 5 easy LeetCode problems: Two Sum, Valid Parentheses, Reverse a String, FizzBuzz, Contains Duplicate
 - Q: Why is a dictionary lookup $O(1)$ while searching an unsorted list is $O(n)$?
 - Note: keep doing 1–2 easy problems a week going forward, don't stop after this week — many junior screens include one easy/medium DSA question
-

PHASE 3: FastAPI Core (Weeks 7–14)

Week 7 — FastAPI Basics + Auto Docs + Health Checks

Concepts: First app, path/query parameters. **Also:** `/docs` and `/redoc` — this is FastAPI's biggest selling point, learn to customize descriptions, tags, and response examples in the OpenAPI schema. **Also:** a `/health` endpoint — load balancers and container platforms ping this to know if your app is alive; without one, a fine app can get killed for looking "unresponsive" **Test:**

- Build 3 endpoints, then customize their `/docs` entries with descriptions and example responses
- Add a `GET /health` endpoint returning `{"status": "ok"}`
- Q: Why does auto-generated interactive documentation matter to an employer evaluating your API?
- Q: What's the difference between a liveness probe and a readiness probe? (Liveness = is the app crashed? Readiness = is it ready to receive traffic?)

Week 8 — Request Bodies & Pydantic

Concepts: Pydantic models, validation, optional fields, nested models, response models **Test:**

- `/users` POST endpoint accepting a Pydantic model, returning it with a generated ID
- Q: Why Pydantic instead of raw JSON parsing?

Week 9 — Full CRUD (In-Memory)

Concepts: GET/POST/PUT/DELETE, HTTPException, status codes **Test:**

- Full CRUD API for a "tasks" resource, in-memory storage
- Q: 404 vs 400 — when does each apply?

Week 10 — Dependency Injection & Background Tasks

Concepts: `Depends()`, reusable dependencies, **BackgroundTasks** for offloading work outside the request-response cycle (e.g. sending a notification after creating a resource) **Test:**

- Refactor tasks API to use a shared dependency, add a background task that logs every task creation to a file
- Q: Why shouldn't a slow operation (like sending an email) block the API response?

Week 11 — Real Async: Concurrency Done Right

Concepts: The truth about async — `async def` does NOT automatically make things faster. It only helps with I/O-bound waiting, and can make things *slower* if you run CPU-bound work inside it. Learn `run_in_threadpool` for sync-blocking code, and `asyncio.gather` + `httpx` for genuinely concurrent external calls **Test:**

- Build an endpoint that calls 3 external APIs (e.g. weather, crypto price, a joke API) concurrently using `asyncio.gather`, and compare its response time against calling them sequentially
- Q: If async doesn't automatically speed things up, what specifically does it speed up?

Week 12 — Authentication

Concepts: JWT concepts, OAuth2PasswordBearer, password hashing (passlib), issuing/verifying tokens **Test:**

- `/login` endpoint issuing a JWT, protect one existing endpoint
- Q: Why never store plain-text passwords?

Week 13 — Middleware, CORS, Resilience & Logging

Concepts: Custom middleware, CORS config, global exception handlers using `logging` (or `structlog`) instead of `print` — log request IDs, timestamps, and stack traces to stdout so cloud platforms can ingest them. **Also:** rate limiting with `slowapi`, retry logic with `tenacity` — real production concerns, especially once you're calling paid external APIs (like an LLM) that cost money per request **Test:**

- Add CORS config, a global error handler that logs via `logging.error()` (not `print`), rate-limit one endpoint, and wrap one external API call with a retry policy
- Q: Why does an unprotected LLM-wrapping endpoint risk an unbounded bill?
- Q: Why is `print()` bad for production logging, and what should you use instead?

Week 14 — Testing, Linting & Formatting

Concepts: pytest, FastAPI's TestClient, fixtures. **Also:** `ruff` (or `black` + `flake8`) for auto-formatting and linting — real teams reject inconsistently formatted PRs, and automation removes the argument entirely **Test:**

- 5+ tests covering your tasks API including at least one error case
- Install `ruff`, run it against your codebase with `ruff check . --fix`, **review the diff before committing** (auto-fixers can occasionally break logic or import the wrong thing — never trust them blindly), and optionally set up a pre-commit hook so unformatted code can't be committed
- Q: Why test error cases, not just success cases?
- Q: Why do teams enforce automated formatting rather than debating style in every PR?

Phase 3 Checkpoint: Auth-protected, rate-limited, tested tasks API with background tasks and documented `/docs`. **Start actively applying to freelance gigs from this point** — don't wait for the database phase.

PHASE 4: Database, Caching & Deployment (Weeks 15–20)

Week 15 — SQL Fundamentals

Concepts: SELECT, JOIN types, GROUP BY, aggregates, schema design **Test:** 10 queries against a sample dataset. Q: INNER JOIN vs LEFT JOIN?

Week 16 — SQLAlchemy 2.0 (Async), Connection Pooling & SQL Injection Safety

Concepts: Why SQLAlchemy over SQLAlchemy for job-readiness — it's the industry-standard ORM most companies actually run. Async session setup, `select()` statement syntax, defining models. **Also:** connection pooling — `create_async_engine(..., pool_size=10, max_overflow=20)` and why the defaults aren't enough once real traffic hits your API. **Also:** how SQLAlchemy parameterizes queries automatically, and the specific way developers accidentally bypass that protection **Test:**

- Recreate your tasks model as a real SQLAlchemy 2.0 table connected to Postgres, with an explicit `pool_size/max_overflow` configured
- Q: What's a downside of relying on an ORM too heavily?
- Q: What happens if 100 users hit your API simultaneously and your DB pool only has 5 connections?
- Q: How does SQLAlchemy protect against SQL injection, and in what edge case might a developer accidentally bypass that protection? (Hint: raw `text()` calls or string concatenation)

Week 17 — Full CRUD with Real Database

Concepts: Database sessions, Alembic migrations, replacing in-memory logic with real queries **Test:**

- Fully migrate tasks API to Postgres — persists across restarts
- Note the deployment rule for later (Week 20): migrations run **before** the new app code starts — if the app boots against a schema that doesn't match its models yet, it crashes
- Q: Why use migrations instead of editing the schema directly?
- Q: What happens if you deploy new app code but forget to run migrations first? (SQL errors — the schema doesn't match the ORM models)

Week 18 — Relationships & Redis Caching

Concepts: One-to-many/many-to-many relationships, foreign keys. **Also:** Redis via `aioredis` — caching repeated GET queries, token blacklisting for logout **Test:**

- Add a related "categories" resource (one-to-many with tasks), cache the tasks-list GET endpoint in Redis, and implement JWT blacklisting on logout
- Q: Why is querying the database on every single GET request wasteful once traffic grows?

Week 19 — Docker & Docker Compose

Concepts: Dockerfile for your FastAPI app, **docker-compose.yml** wiring together app + Postgres + Redis for local dev — a single Dockerfile alone isn't enough once you have real dependencies **Test:**

- **docker-compose up** should spin up your full stack (app, DB, Redis) with one command
- Q: Why is containerizing Postgres locally better than pointing at a shared dev database?

Week 20 — Deployment & Config Management

Concepts: Environment variables, deploying with docker-compose support (Railway or Fly.io), connecting a hosted Postgres + Redis instance. **Also:** **pydantic-settings** (or **python-dotenv**) as the actual industry-standard way to load config — not just "put it in an env var," but validated config loading that fails loudly if something's missing. **Also:** environment separation — local/staging/production need different DBs, keys, and debug flags, or you risk running production against your local database **Test:**

- Full stack deployed live with a working public URL
- Use **pydantic-settings** to load your Postgres/Redis URLs and JWT secret from a **.env** file, with validation that raises a clear error if a required variable is missing
- Extend it with an **ENVIRONMENT** variable (**local/test/production**) that loads the matching **.env.*** file, with **DEBUG=True** only locally and **docs_url=None** in production
- Also load **ALLOWED_ORIGINS** as a comma-separated env var (not hardcoded) and pass the parsed list to **CORSMiddleware** — so a deployed frontend on a new domain doesn't need a backend code change to stop being CORS-blocked
- Q: Why should DB credentials never be hardcoded?
- Q: What happens if you accidentally deploy to production with **DEBUG=True**? (It exposes stack traces and can leak sensitive config through the **/docs** panel)

Week 21 — Mock Interview Week

Before building portfolio projects, prove you can *explain* what you built, not just that it runs.

- Record yourself (phone camera is fine) answering out loud: Why did you choose async vs sync in a specific endpoint? How do database migrations work and why do you need them? How do you secure a JWT end-to-end? What does your caching strategy protect against?
 - If you stumble or can't explain it without notes, that's a signal to revisit that week's material — not a reason to panic
 - **Also do this week:** find one other developer (a Discord server or a subreddit for learners works fine) and do a 30-minute live screen-share explaining your code to each other. Someone else asking "why did you do it that way?" while looking at your screen exposes gaps recording yourself never will
 - **Also start this week:** add a "Daily Standup Notes" section to every project README going forward — short entries like *"Day 14: Fixed N+1 query issue in tasks endpoint"*. This is a small habit that signals real engineering thinking to anyone reviewing your GitHub.
 - **Before shipping Project 1, run this checklist:**
 - Does your app fail fast with a clear error if a `.env` variable is missing?
 - Does `/docs` disappear in production (`docs_url=None` when `ENVIRONMENT=="production"`)?
 - Does your app handle shutdown gracefully — closing DB/Redis connections cleanly via a shutdown event — instead of dropping in-flight work?
-

PHASE 5: Portfolio Projects (Weeks 22–27)

Weeks 22–23 — Project 1: SEO Metadata & Audit API (concrete spec)

Build an API that: accepts a CSV of URLs (file upload), processes them as a **background job** (not blocking the request), scrapes/extracts SEO metadata (title tags, meta descriptions, headers — via `beautifulsoup`, or the Google Search Console API if you want to go further), stores results in Postgres, and exposes a status endpoint the client polls (or a webhook) to check job completion. **Make it bulletproof, not naive:** use `httpx.AsyncClient` (not plain `requests.get`) with explicit timeouts and a real `User-Agent` header — a naive scraper gets your IP blocked fast. Add a fallback to `og`: meta tags when standard title/description tags are missing. **Why this spec:** it proves file uploads, background jobs, real HTTP client handling, and real data processing — not just a toy CRUD wrapper — while leaning directly on your SEO domain knowledge.

Weeks 24–25 — Project 2: CRUD + Auth Showcase

Clean, tested CRUD API with full auth, relationships, caching, and a proper test suite — the project that proves general backend competence independent of any niche use case.

Weeks 26–27 — Project 3: LLM-Integrated API (concrete spec)

Don't just wrap a chat endpoint — that's a 1-hour tutorial and won't stand out. Build: a **streaming** response endpoint (`StreamingResponse`) plus **function/tool calling** — e.g. an agent that can call your own database or a weather API as a "tool" mid-conversation. Add rate limiting and retry logic (from Week 13) since this endpoint costs real money per call.

Final Checkpoint: All 3 projects live, dockerized, tested, with Daily Standup Notes in each README, linked from a personal portfolio page.

Rules

1. Don't skip a Week Test even if confident — the hands-on part catches gaps quizzes miss
2. Commit to Git daily — this is your visible proof-of-work, not optional
3. If a test takes more than 2 extra days, slow down rather than pushing forward with a shaky foundation